

기획, 개발부터 배포까지

사이드 프로젝트 실물 금 자산 관리 서비스 ‘금장부’



백정민 | backshren20@naver.com

Index

목차소개

- 01 프로젝트 소개
- 02 프로젝트 개발 A to Z
- 03 트러블 슈팅
- 04 주요 성과 요약 (개인 성장 중심)
- 05 향후 개선 사항 및 확장 계획

01 프로젝트 소개

추진 개요



제가 2년 넘게 꾸준히 이어오고 있는 취미이자 재테크 수단은 ‘콩알금 모으기’입니다. 최근 안전자산인 금에 대한 수요가 증가하면서, 보유 중인 실물 금의 현재 가치를 체계적으로 관리하고 싶다는 생각이 들었습니다. 이러한 문제의식에서 출발해 실물 금 자산 관리 프로그램을 직접 개발하게 되었습니다.

추진 기간 및 범위



 전체 개발 기간
2026.01.14 ~ 2026.02.10 (약 4주, 28일)

- 기획 → 설계 → 구현 → 배포 → 문서화까지 단독 수행

추진 전략 및 프로젝트 목표



본 프로젝트는 실사용 경험을 기반으로 핵심 기능부터 구현하고, 도메인 중심 설계와 병행 개발을 통해 빠르게 완성도를 높이는 전략으로 추진하였습니다.

- 프로젝트 목표**

1인 프로젝트로 모든 영역을 일정 내에 수행해야 했기 때문에, **핵심 기능(매입·매도 기록, 보유 자산 관리) 범위를 최소화하고 안정적인 배포와 백엔드 개발 역량을 강화하는 것을 목표로** 프로젝트를 수행했습니다.

01 프로젝트 소개 - 기술 스택

구분	기술	사용 범위
Backend	Java, Spring Boot Spring Data JPA MySQL	REST API 설계 및 비즈니스 로직 구현 매입·매도 도메인, LOT 기반 자산 관리 로직 개발 도메인 중심의 엔티티 설계 거래 이력 및 자산 데이터 저장
Frontend	React JavaScript	자산 현황, 매입·매도 화면 구현 백엔드 API 연동 및 상태 관리 컴포넌트 기반 UI 구성 + 데이터 렌더링
Batch	공공데이터 API 연동 Spring Batch / Scheduler	금 시세 데이터 수집 (공공데이터 API) 금 시세 정기 적재 및 자동 갱신
Infra & Deployment	AWS EC2 / Router 53 Nginx	서버 환경 구축 및 운영 React 정적 파일 서빙 (프론트, 백 분리 배포)
Tooling	GitHub IntelliJ / VS Code	코드 이력 관리 백엔드 / 프론트엔드 개발 환경

요약

- 백엔드는 기존에 익숙한 Java, Spring Boot를 기반으로 프로젝트를 구성하고, 도메인 중심 설계와 데이터 영속성 관리를 강화하기 위해 JPA를 적용했습니다.
- 프론트엔드는 필요에 따라 AI를 적극 활용해 직접 구현했습니다. 이전에 [인프런 - React 강의] 수료 경험을 바탕으로 React를 주요 기술로 선택했습니다. 단일 페이지 구조 (SPA)로 여러 컴포넌트를 구성하고, 백엔드 API와의 연동을 통해 데이터 중심 UI를 구현했습니다.
- 공공데이터 API 를 연동해 일별 금 시세 데이터를 수집 및 배치 스케줄링으로 일 1회 정기 적재하여 현재 보유 금 자산 가치를 자동 갱신하여 보여줍니다.
- 운영 환경은 AWS EC2를 사용해 구축했으며, 배포 및 도메인 관리를 고려해 AWS Route 53을 활용해 DNS를 설정했습니다.

02 프로젝트 개발 A to Z - 기획

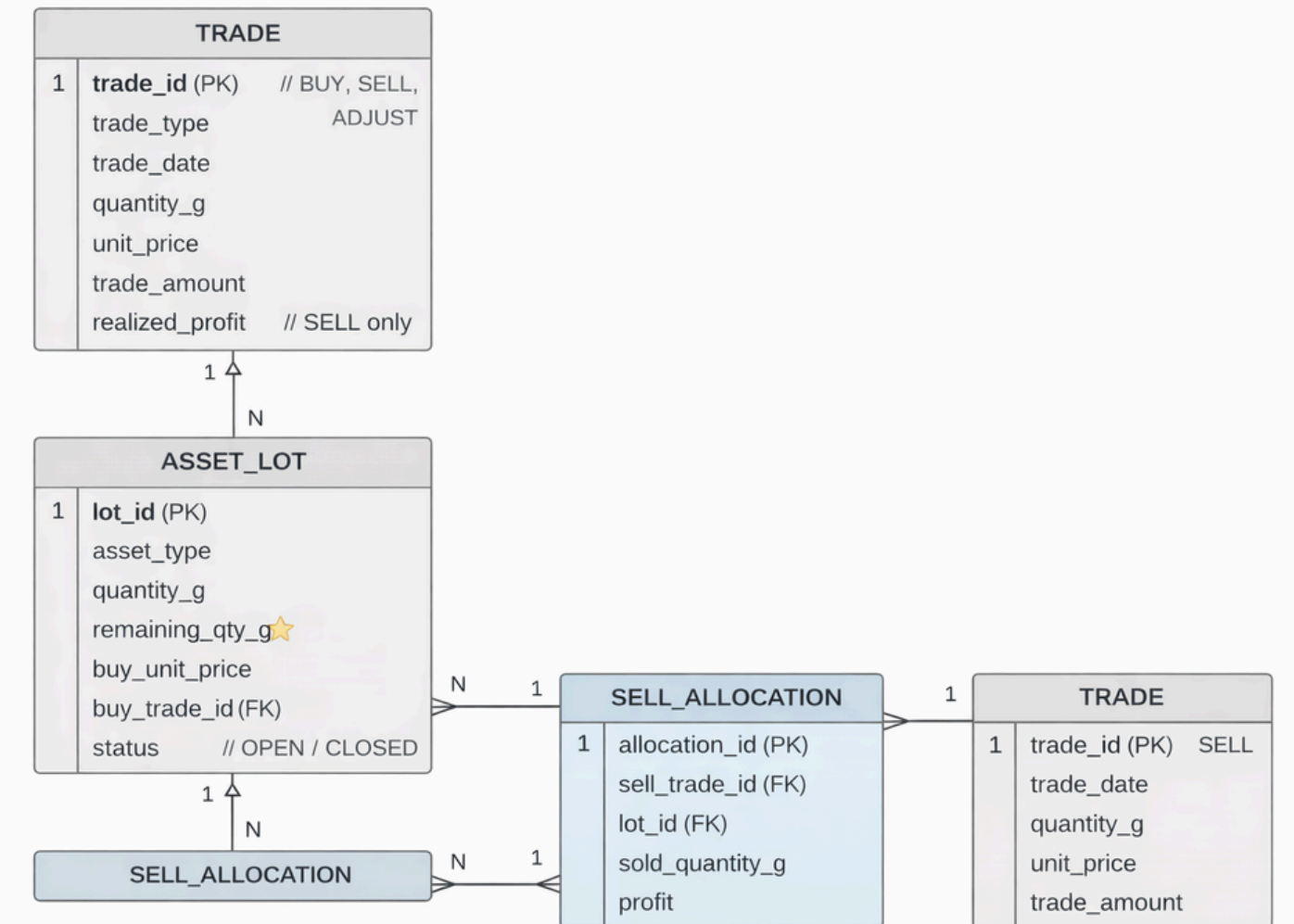
- 기획 단계에서 프로젝트는 ‘일반 가계부와 금 투자 기록’ 을 함께 관리할 수 있는 프로그램으로 개발 방향을 잡았습니다. 주요 기능인 ‘금 투자 기록’은 월간 캘린더에 스탬프 형식 (ex. 깃허브 커밋보드) 처럼 금 투자가 발생한 일자에 하이라이트 표시와 세부 사항을 볼 수 있는 list 목록을 주도로 설계했습니다.
- 그러나, 일반 가계부와 금 투자 기록을 함께 가져가니 프로젝트 주제가 모호해지는 것을 느껴, 주기능인 **금 투자 기록만 관리하는 프로그램으로 개발 방향**을 재설정했습니다. 이때 핵심 기능(매입·매도 기록, 보유 자산 관리) 중심으로 범위를 최소화하였습니다.
- 또한, 프로젝트 핵심 주제인 ‘현재 나의 실물 금 자산 가치가 얼마인지’ 기준점이 되는 과거 n년부터 최근까지 수집 된 금 시세 데이터를 제공하고, 기존 스탬프 캘린더가 아닌 시세 차트 내에서 매입/매도 기록을 볼 수 있는 보드를 제공하기로 기획을 확정했습니다.
- 이 프로그램은 ‘실물 형태가 있는 금’ 만을 자산으로 취급하도록 한정하고, 주식 거래와 유사한 구조 (LOT 기반 관리) 로 모델링하였습니다.



위) 금 납입 캘린더 (초기)
오른쪽) 금 시세 차트 기획

02 프로젝트 개발 A to Z - 도메인 설계

- 도메인 설계에 앞서 ‘실물 금’ 거래 방식에 대한 정의가 필요했습니다.
- 프로젝트 초기에는 금 투자 기록을 입출금 서비스와 유사하게 바라보고, ‘총 보유 수량(g)’ 단위로 매입·매도 로직을 설계했습니다. 사용자가 직접 ‘몇 g’을 ‘얼마에’ 매입·매도하는 구조를 가정했고, 매도가 발생할 경우 FIFO 원칙에 따라 ‘먼저 매입된 금 수량’에서 차감되도록 구현하고자 했습니다.
- 그러나 설계를 구체화하는 과정에서, 실물 금은 형태와 무게가 현실에서 고정된 자산이라는 점이 **일반적인 금융 자산과 본질적으로 다르다는 점**을 인식하게 되었습니다. 단순히 총 보유 수량에서 매도가 발생하는 구조는, 실제로 어떤 ‘실물 금’을 매도했는지에 대한 추적이 불가능하다는 한계가 있었습니다.
- 이에 따라 매도 방식을 개선하여, 사용자가 직접 매도 대상이 되는 ‘실물 금’을 선택하도록 하고, 매입 기록(묶음 단위, lot)을 기준으로 매도가 이루어지는 구조로 설계를 변경했습니다. 이를 통해 실물 금의 물리적 특성을 도메인 모델에 반영하고, 각 매입 건 단위의 보유 수량 및 매도 이력을 명확하게 관리할 수 있도록 했습니다.



02 프로젝트 개발 A to Z - 백엔드

REST API 설계 및 비즈니스 로직 구현



- REST API 기반의 실물 금 자산 관리 서버 구현
- 매입·매도, 보유 자산, 요약 정보 등 핵심 비즈니스 로직 개발
- 매입·매도 이력, LOT(매입 묶음) 관리 구조 구현
- 매도 수량 초과, 잔여 수량 부족 등 예외 케이스 처리

금 시세 데이터 적재 배치 개발



- 공공데이터 API (금 시세 데이터) 활용하여 n년 치 실물 금 시세 데이터 수집용 배치 개발
- Spring Boot Batch, Schedule 활용하여 서버에 하루 1회 시세 데이터 적재하도록 스케줄 등록

“백엔드에서 제일 고민한 점은?”

- 가장 고민했던 부분은 ‘실물 금 거래 데이터의 정합성을 어떻게 유지할 것인가’였습니다. 특히 매입·매도 이력이 시간순으로 누적되는 구조에서, 과거 거래가 수정되거나 입력 오류가 발생했을 때도 전체 자산 상태가 일관되게 계산되도록 설계하는 것이 핵심 과제였습니다.
- 이를 위해 단순히 현재 보유 수량만 저장하는 방식이 아니라, LOT(매입 묶음) 기반 구조를 도입해 매입 단위를 명확히 관리하고, 매도 시에는 서버에서 매도 가능 수량이 있는 실물 금 데이터를 불러와 LOT 단위로 수량을 차감하도록 구현했습니다. 이 과정에서 매도 수량 초과, 잔여 수량 부족 같은 예외 케이스를 트랜잭션 단위에서 검증하도록 설계해 데이터 불일치를 방지했습니다.

☞ 이 경험을 통해 단순 CRUD 구현이 아니라, 도메인 특성을 반영한 데이터 모델링과 서버 책임 설계의 중요성을 깊이 고민하게 되었습니다.

02 프로젝트 개발 A to Z - 프론트엔드



- SPA 구조로 화면 전환 없이 자산 현황 조회, 매입·매도 등 핵심 기능을 한 페이지에서 처리하도록 UI 설계
- 백엔드 REST API 응답 구조를 기준으로 컴포넌트를 분리하고, 데이터 흐름을 고려한 컴포넌트 단위 모듈화
- 사용자 입력 빈도가 높은 매입·매도 기능은 모달 UI로 분리해 UX 흐름을 단순화

- 백엔드 개발자로서 전체 시스템을 이해하기 위해 프론트를 학습했습니다. 0 에서 부터 시작한 프론트 화면을 점진적으로 구현해가며, 백엔드와 프론트엔드 분리 환경, 라이프사이클 등 웹 애플리케이션 동작 원리를 이해했습니다.

👉 이 경험을 통해 이후 백엔드 API를 설계할 때 “프론트에서 쓰기 쉬운 구조인가?”를 함께 고민하게 되었습니다.

“프론트엔드 개발 경험 회고”

- 98강으로 구성된 React 온라인 강의를 수강한 이후, 실제 프로젝트에 프론트엔드를 직접 적용하는 데 도전했습니다. JavaScript 자체에는 익숙했지만, 컴포넌트 단위로 상태와 화면이 연결되는 React의 구조를 이해하는 데 상당한 시행착오를 겪었습니다.
- 이벤트 핸들링, 상태 변경에 따른 렌더링 흐름, API 연동 과정에서의 CORS 설정, 그리고 useEffect 의 의존성 배열을 통한 실행 시점 제어 등은 접해본 개념들이었지만 실제 구현 과정에 적용하는데 어려움을 겪어 생성형 AI를 활용했습니다.
- 다만 단순히 생성형 AI에 전적으로 의존해 프론트를 구현하는 수준에 그치지 않고, **프론트엔드 전체 구조와 데이터 흐름은 직접 설계**하고, 차트, 슬라이더, 자산 카운팅 효과 등 표현 중심 기능만 도움을 받아 학습하며 점진적으로 화면을 구현했습니다.
- 그 결과, 백엔드에서 설계한 REST API를 기반으로 데이터 중심 UI를 직접 구현할 수 있었고, 프론트엔드와 백엔드 간 데이터 흐름과 책임 분리를 이해하는 계기가 되었습니다.

03 트러블 슈팅 - 1

도메인



UI 기능은 많았지만 핵심 도메인이 보이지 않는 문제

캘린더, 차트, 슬라이더 등 기능은 구현했지만 “장부 서비스의 핵심 기능”이 UI에서 드러나지 않음

☞ 원인 : 기능 단위 구현에 집중해 사용자 흐름 관점 설계 부족



전체 레이아웃 재설계
매입은 모달, 상세 조회는 페이지 분리하는 UX 채택

도메인



매입·매도 도메인 설계의 근본적인 어려움

기존 도메인은 FIFO / 평균단가 변경 시 남은 수량 전체 재계산이 필요한 구조였음

☞ 실물 금 특성상 자산 단위 관리 필요



매도는 기본적으로 자산 단위로 기준 잡기
매입/매도 기록은 절대 수정하지 않고 누적시키고 총 자산은 항상 거래 기록 기반으로 계산

프론트엔드



React useEffect 사용 미숙으로 인한 렌더링 제어 문제

API 호출, 슬라이더, 카운팅 효과 등에서 불필요한 리렌더링 발생

☞ 원인 : useEffect 의존성 배열과 라이프사이클 이해 부족 + state가 비동기처럼 동작하는 특성에 대한 경험 부족



기능별로 리렌더링 기준 명확히 정의
상태 관리와 사이드 이펙트 제어 개념 학습 및 적용

03 트러블 슈팅 - 2

프론트엔드



React-Spring 연동 과정에서의 CORS 문제

React(localhost:5173)와
Spring(localhost:8080) 연동 시 CORS 오류
로 인해 API 호출 실패 발생

👉 원인 : 화면 요청과 API 요청을 동일하게
CORS 허용



/api/** 경로에 대해서만 CORS 허용
화면 요청에는 CORS 미개방

백엔드



공공데이터 API 연동 중 인코딩 이슈

공공데이터 API 호출 시 지속적인 401 오류 발생

👉 원인 : WebClient에서 서비스키 이중 인코딩 발생으로 생긴 문제



Encoding key + build() 사용 시
build(true) 로 인코딩 중복 방지
서비스키를 서버 환경변수로 분리해 사용

백엔드



JdbcTemplate 기반 레이어 구조 혼란

A초기 JdbcTemplate 기반 백엔드 구현 시 Controller →
Service → Repository 구조를 의도했으나, 실제로는
Repository가 JdbcTemplate을 직접 의존하면서 혼란 발생

👉 원인 : JdbcTemplate 사용 경험, “데이터 접근 계층 분리”
에 대한 이해 부족

Repository 생성 순서 및 의존성 흐름에 대한 설계 미흡



Repository 계층만 JdbcTemplate 의존하도록 분리
이후 JPA 도입을 고려해 도메인 중심 설계 방향으로 전환

03 트러블 슈팅 - 3

백엔드



JPA 도입 이후 @Query 사용에 대한 고민과 적용

JPA 도입 초기에 조회 로직 구현 과정에서 엔티티 연관관계를 활용한 객체 중심 조회에 한계 느낌

👉 원인 : 자산 관리 도메인의 특성상 총 보유 수량, 누적 매입 금액, 손익 등 집계·계산 쿼리가 많았음
메서드 네이밍 쿼리로 표현하니 가독성과 의도가 불명확해지는 문제 발생

JPA 메서드 네이밍 쿼리의 한계를 보완 필요성 체감



계산·집계가 필요한 조회 로직에 한해 JPQL 기반 커스텀 조회 쿼리(@Query)로 분리해 도입
쿼리 의도를 명확히 드러내는 방식으로 작성

문자열 기반 쿼리이기 때문에 엔티티 구조 변경 시 런타임 오류 가능성 존재 (실제 개발 과정에서 겪음)
그러나, 계산 기준이 코드상에 드러나 도메인 규칙 이해 및 디버깅이 수월해짐

운영 배포



AWS EC2 수동 배포의 불안정성

EC2 재부팅 시 애플리케이션이 중단되어
nohup 기반 실행의 한계



systemd 서비스로 Spring Boot 등록
EC2 재부팅 시 자동 실행 구조 구성

04 주요 성과 요약 (개인 성장 중심)



프론트엔드-백엔드 데이터 흐름을 고려한 설계로 안정적인 배포를 완료하고,
시스템 전반을 바라보는 문제 해결 역량을 강화했습니다.

프로젝트 초기 목표와 성과 점검

구분	목표치	평가 (5점 만점)	성과
프로젝트 완성도 관점	초기 기획에서 정의한 기능들은 구현했는지 사용자 관점에서 실 사용에 문제 없는지 기획부터 배포까지 성공했는지	★★★★	1) 제한된 기간 내 핵심 기능(매입·매도, 자산 관리)에 집중하여 구현을 완료했으며, 부가 기능은 운영 안정성과 확장성을 고려해 개선 과제로 분리했습니다. 2) 실제 사용을 기준으로 기능을 검증하고, 기획부터 배포까지 프로젝트 전 과정을 단독으로 완주했습니다.
개발 역량 성장 관점 (프론트엔드·백엔드)	백엔드 도메인 설계 및 데이터 정합성 구현 프론트엔드 학습 및 API 연동 성공했는지 새로운 기술 도입은 성공적이었는지 시스템 구조를 이해하며 개발했는지	★★★★★	1) API 데이터가 화면에서 어떻게 구성되고 전달되는지 이해 2) 실무에서 프론트 파트와 협업 시 개발 흐름을 빠르게 파악하고 원활한 커뮤니케이션이 가능해졌습니다!
운영/배포 및 실서비스 관점	AWS 기반 실제 배포 환경 구축에 성공했는지 배치, 로그 등 운영 요소 고려했는지	★★★★	1) AWS 기반의 실제 운영 환경에 애플리케이션 배포 과정 경험 2) 배치 스케줄링과 로그 확인 등 운영 단계에서 필요한 요소들을 고려한 서비스 환경을 구축할 수 있게 되었습니다.

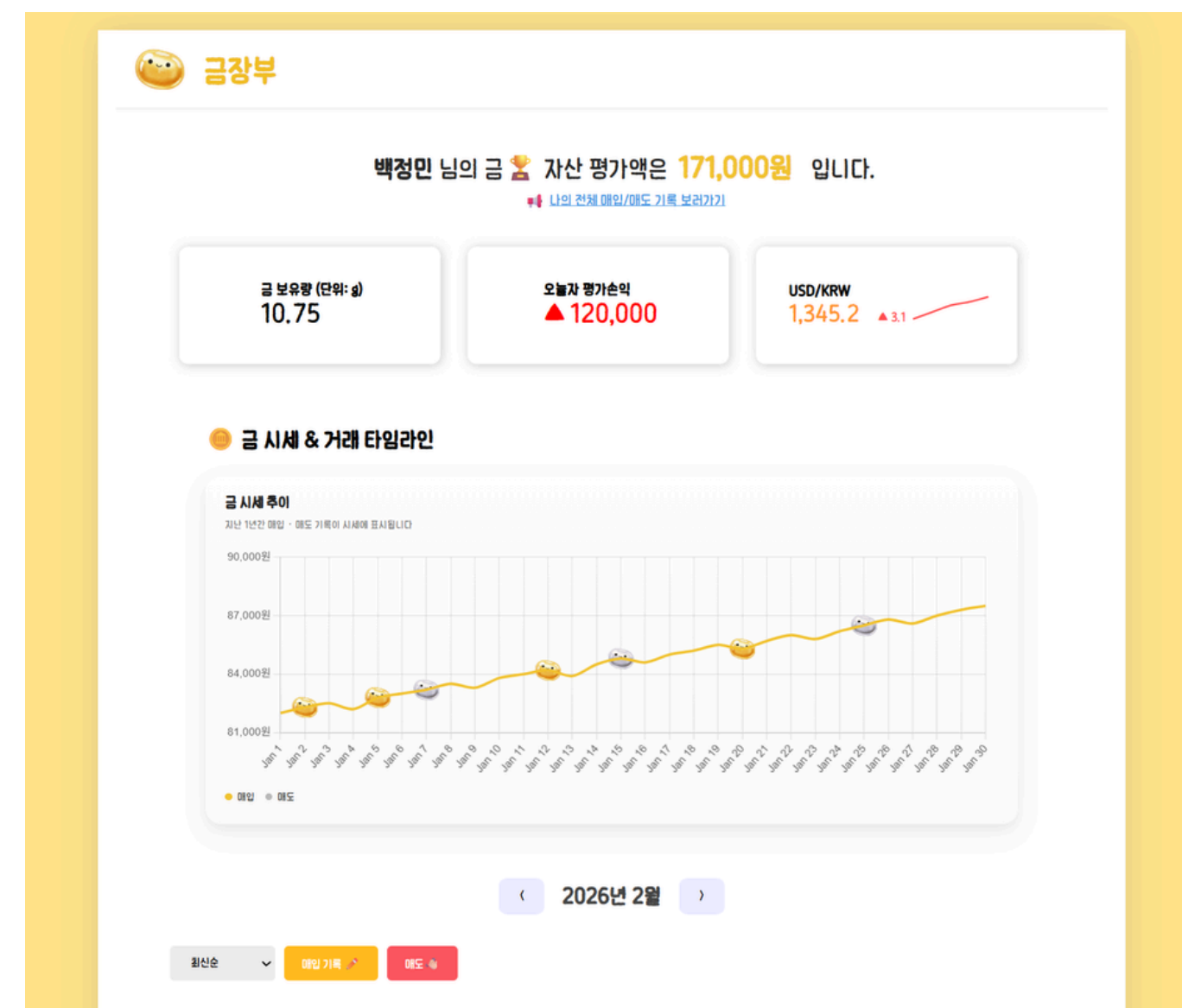
05 향후 개선 사항 및 확장 계획

프로젝트 고도화 및 운영 환경 개선 방향

현재 PC 중심 UI 를 모바일 환경에서도 사용 가능하도록 반응형 UI 로 개선할 계획입니다. 매입·매도, 자산 조회 등 핵심 기능을 모바일 UX 기준으로 재설계 하여, 향후 모바일 애플리케이션 개발을 학습할 때 참고 가능한 프로젝트 소스로 활용하고자 합니다.

또한 현재 단일 사용자(본인 데이터) 기준으로 구성된 구조에서 회원가입 및 로그인 기능을 포함한 다중 사용자 환경으로 확장할 예정입니다. 사용자별 자산과 거래 이력을 분리해서 관리하고 이를 위해 인증 기반 API 접근 방식 도입을 검토하고자 합니다.

마지막으로 현재 수동으로 이뤄지는 배포 구조를 개선하기 위해 Docker 기반 배포 환경을 표준화하고 위에 내용이 반영될 정도로 서비스가 확장된 후엔 Kubernetes 를 활용해 애플리케이션 장애 발생 시 자동 복구 및 무중단 배포가 가능한 구조로 확장하는 것을 목표로 하고 있습니다.



**봐주셔서
감사합니다.**



백정민 | backshren20@naver.com